# COL7378: Principles of Autonomous Systems
# Semester II, 2025-26

## Reinforcement Learning: Introduction

**Rohan Paul**

# Outline

- Last Class
  - Markov Decision Processes
- This Class
  - An introduction to Reinforcement Learning
- Reference Material
  - Please follow the notes as the primary reference on this topic.

# Acknowledgements

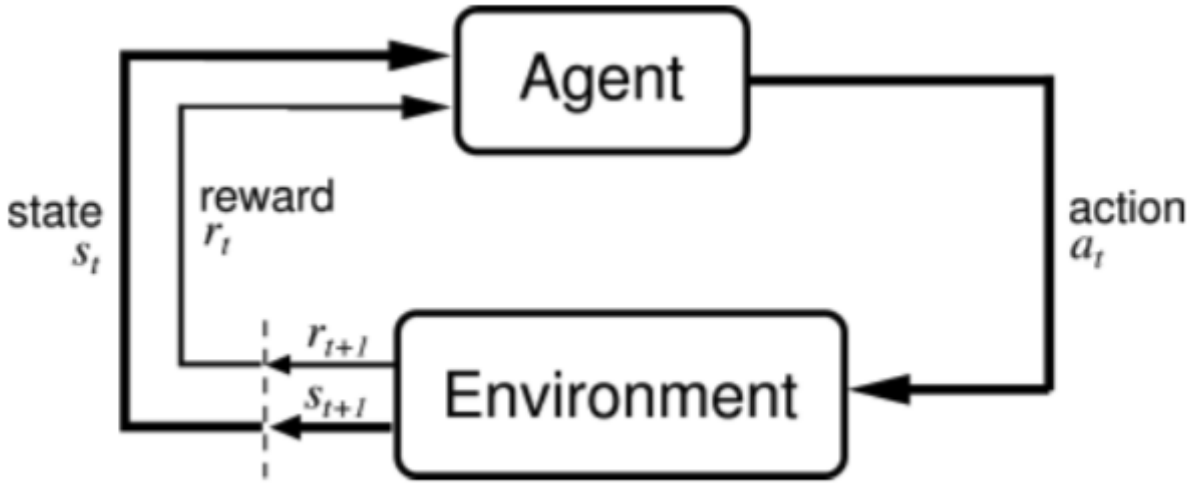
**These slides are intended for teaching purposes only. Some material has been used/adapted from web sources and from slides by Nicholas Roy, Wolfram Burgard, Dieter Fox, Sebastian Thrun, Siddharth Srinivasa, Dan Klein, Pieter Abbeel, Max Likhachev and others.**

# Learning to Act from Interaction/Data

- So far we assumed to have an a-priori model of the domain
  - MDP: Tuple of states, actions, transition function, rewards, start state and discount factor
  - Safe to assume that some parts of the model are constant.
    - State and action sets are fixed and are given by the domain.
- In practice
  - Commonly, we don't know the
    - Transition function (*if I take an action which state I will end up in?*)
    - Reward function (*when I take a transition is it good or bad?)*
  - Experience
    - We can observe transitions and rewards as a function of actions
    - Can we use this data to learn something from this experience?

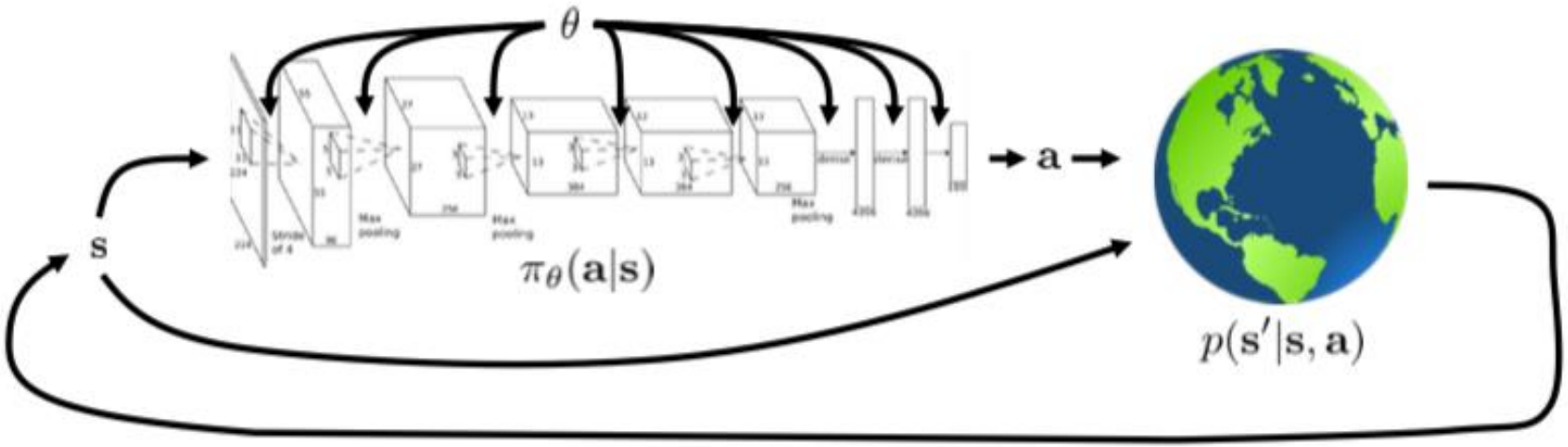
# Learning to Act from Data

- The agent experiences the environment and receives a reward and observes the consequence of the action.
- Does not have the full rewards function or the transition model ahead of time.
- Needs to determine how to act?
- Note: there is evaluative feedback for actions not prescriptive feedback.

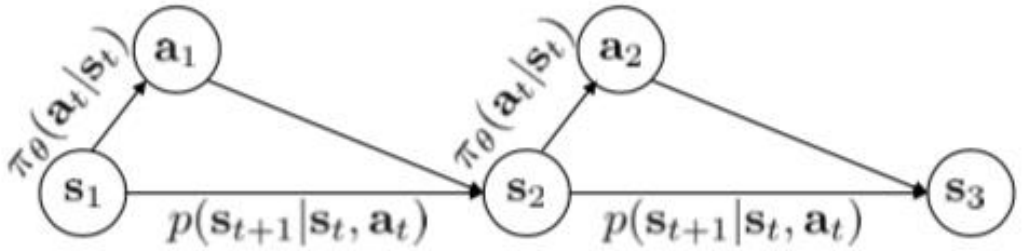



$$\pi^* = \arg\max_{\pi} \mathrm{E}[\sum_{t=0}^{H} \gamma^t R_t(S_t, A_t, S_{t+1})|\pi]$$

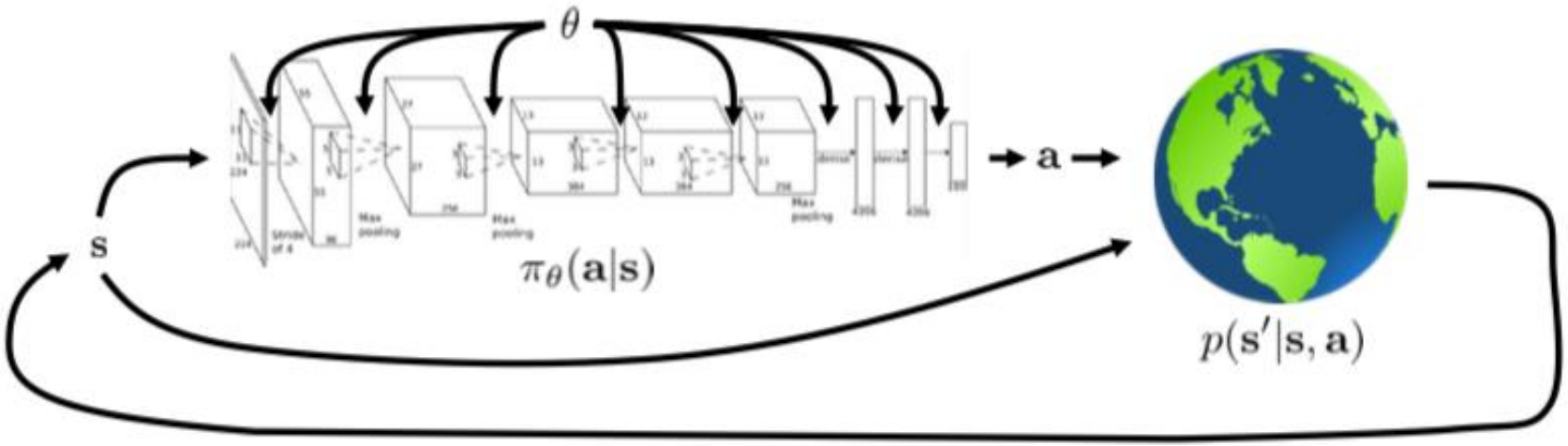
# Goal of Reinforcement Learning




$$\underbrace{p_\theta(\mathbf{s}_1,\mathbf{a}_1,\ldots,\mathbf{s}_T,\mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1)\prod_{t=1}^{T}\underbrace{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)p(\mathbf{s}_{t+1}|\mathbf{s}_t,\mathbf{a}_t)}_{\text{Markov chain on }(\mathbf{s},\mathbf{a})}$$



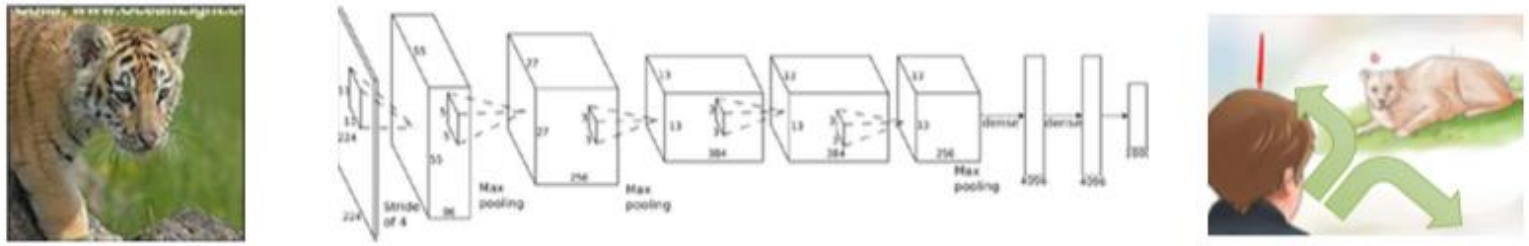
# Goal of Reinforcement Learning




$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[ \sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

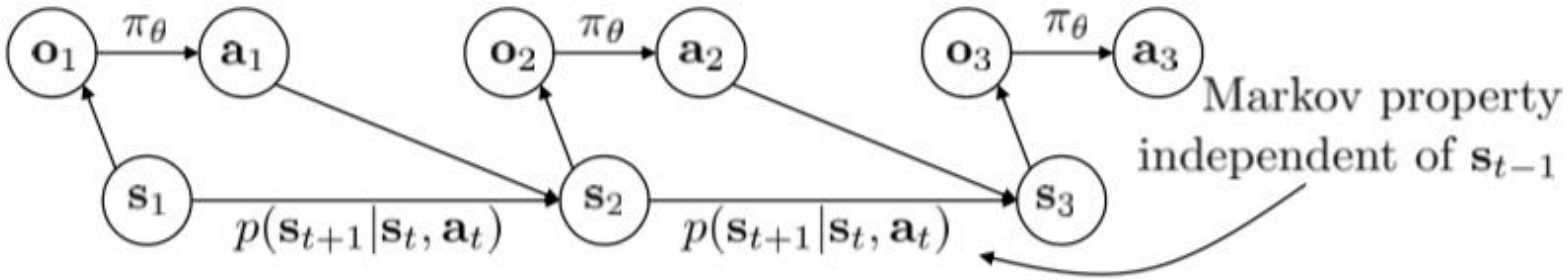
# State Vs. Observation of the State



$\mathbf{o}_t$ $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ $\mathbf{a}_t$

$\mathbf{s}_t$ – state
$\mathbf{o}_t$ – observation
$\mathbf{a}_t$ – action

$\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ – policy
$\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ – policy (fully observed)



# Formalism: *MDP* vs. Partially-Observed MDP

Markov decision process $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{T}, r\}$

$\mathcal{S}$ – state space states $s \in \mathcal{S}$ (discrete or continuous)

$\mathcal{A}$ – action space actions $a \in \mathcal{A}$ (discrete or continuous)

$\mathcal{T}$ – transition operator (now a tensor!)

$r$ – reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$

$r(s_t, a_t)$ – reward

# Formalism: MDP vs. *Partially-Observed MDP*

partially observed Markov decision process $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{E}, r\}$

$\mathcal{S}$ – state space states $s \in \mathcal{S}$ (discrete or continuous)

$\mathcal{A}$ – action space actions $a \in \mathcal{A}$ (discrete or continuous)

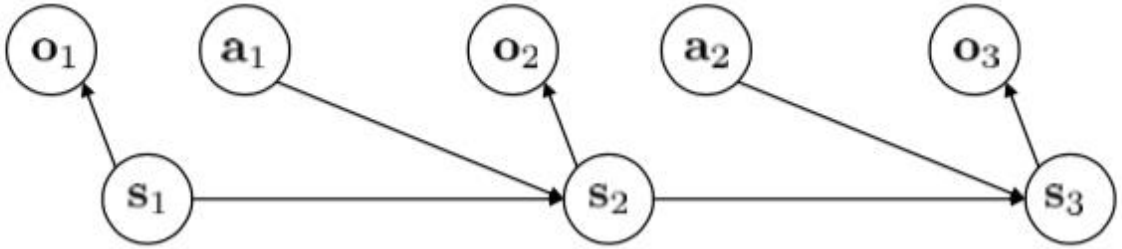
$\mathcal{O}$ – observation space observations $o \in \mathcal{O}$ (discrete or continuous)

$\mathcal{T}$ – transition operator (like before)

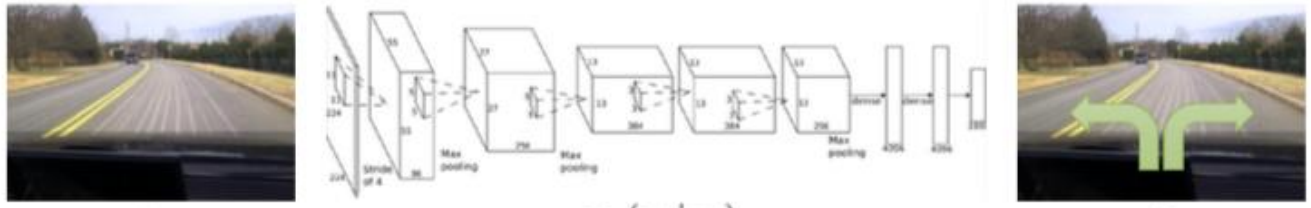
$\mathcal{E}$ – emission probability $p(o_t|s_t)$

**More on POMDPs in a later lecture.**

$r$ – reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$



# Rewards are Obtained After Taking Actions




which action is better or worse?

$r(\mathbf{s}, \mathbf{a})$: reward function

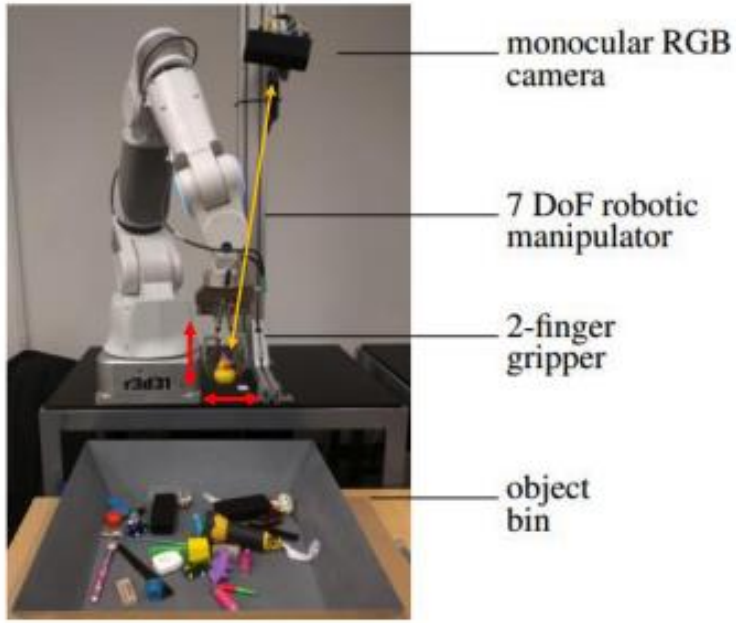
tells us which states and actions are better

$\mathbf{s}$, $\mathbf{a}$, $r(\mathbf{s}, \mathbf{a})$, and $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ define Markov decision process

high reward

low reward

# Example: Grasping

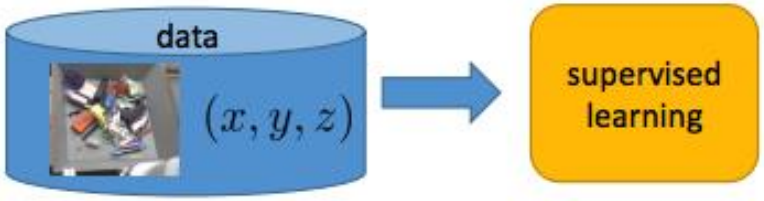


**Option 1:**

Understand the problem, design a solution

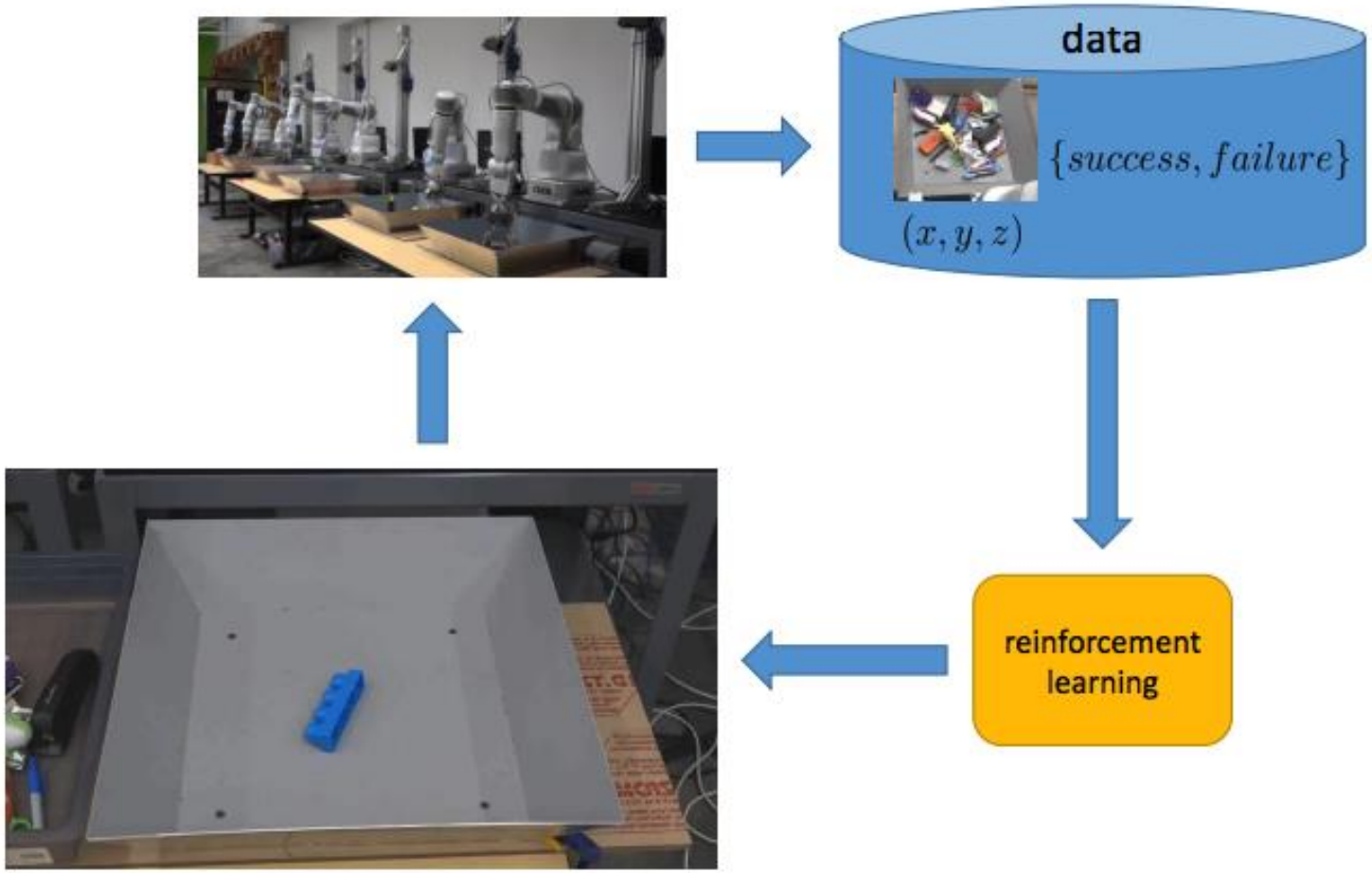
**Option 2:**

Set it up as a machine learning problem



# Example: Grasping

# Example: Grasping

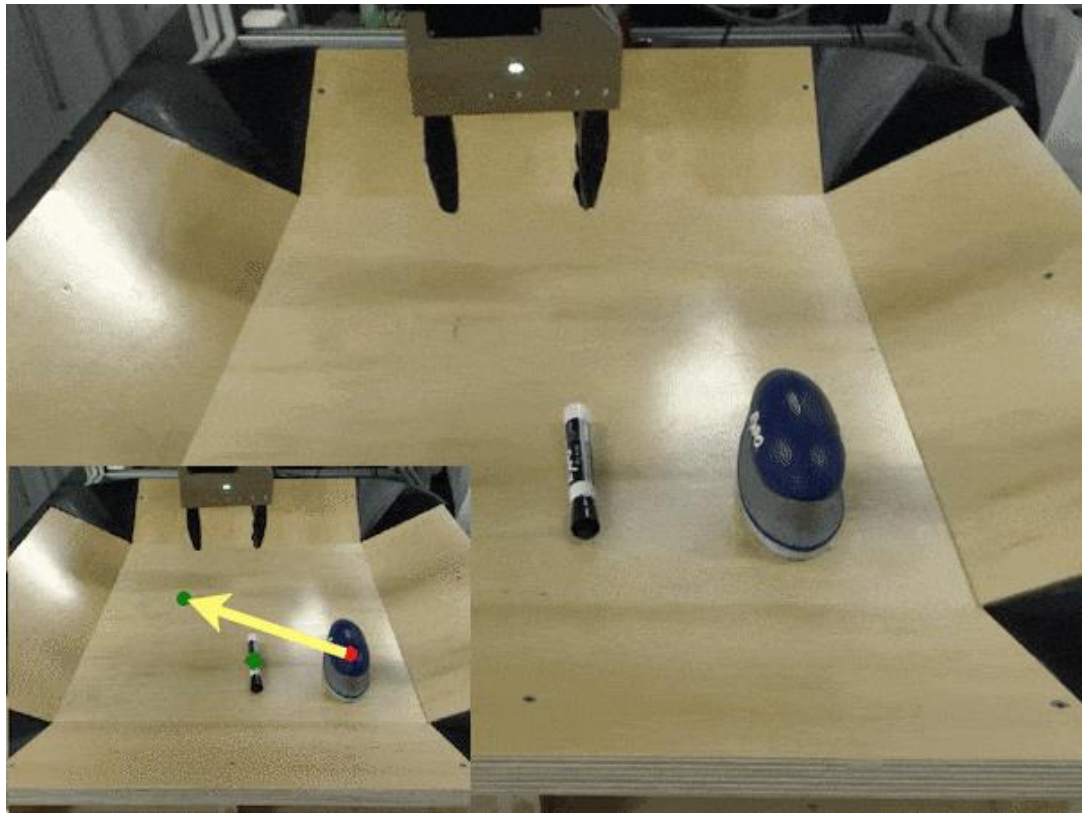



https://sites.google.com/view/qtopt

# Examples: Learning to Manipulate

Chelsea Finn
http://bair.berkeley.edu/blog/2018/11/30/visual-rl/

# Examples: Learning to Manipulate



Chelsea Finn et al.
http://bair.berkeley.edu/blog/2018/11/30/visual-rl/

# Examples: Learning to Walk

Initial

[Kohl and Stone, ICRA 2004]

# Examples: Learning to Walk

Training

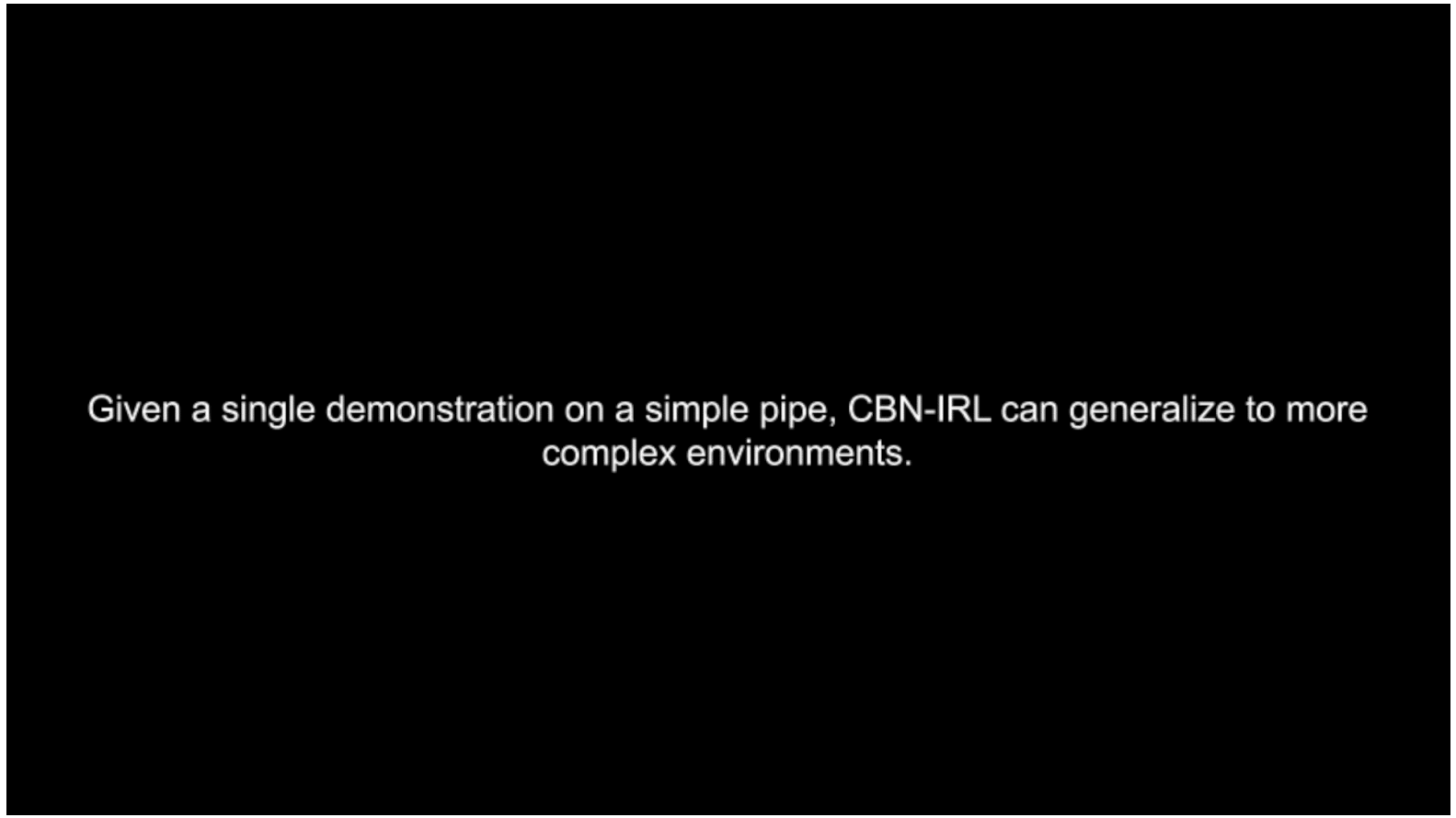
[Kohl and Stone, ICRA 2004]

# Examples: Learning to Walk

Finished

[Kohl and Stone, ICRA 2004]

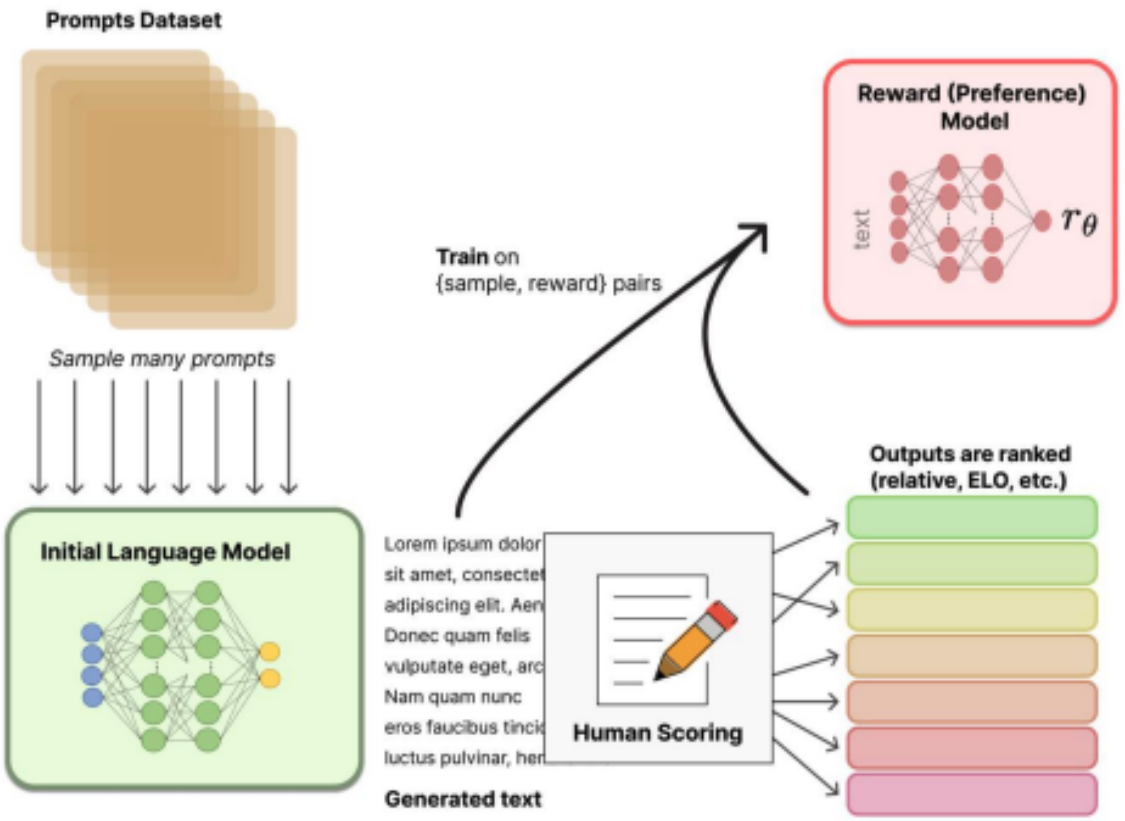
# Examples: Learning to Perform Complex Skills




Inverse RL a form of reinforcement learning

Park, Noseworthy, Paul and Roy. CoRL 2019.

# RL with Language Models




LE

Can you explain to me how RL with human feedback works for language models like I am five?

Sure! So, imagine you have a robot named Rufus who wants to learn how to talk like a human. Rufus has a language model that helps him understand words and sentences.

First, Rufus will say something using his language model. For example, he might say "I am a robot."

Then, a human will listen to what Rufus said and give him feedback on whether it sounded like a natural sentence a human would say. The human might say, "That's not quite right, Rufus. Humans don't usually say 'I am a robot.' They might say 'I'm a robot' or 'I am a machine.'"

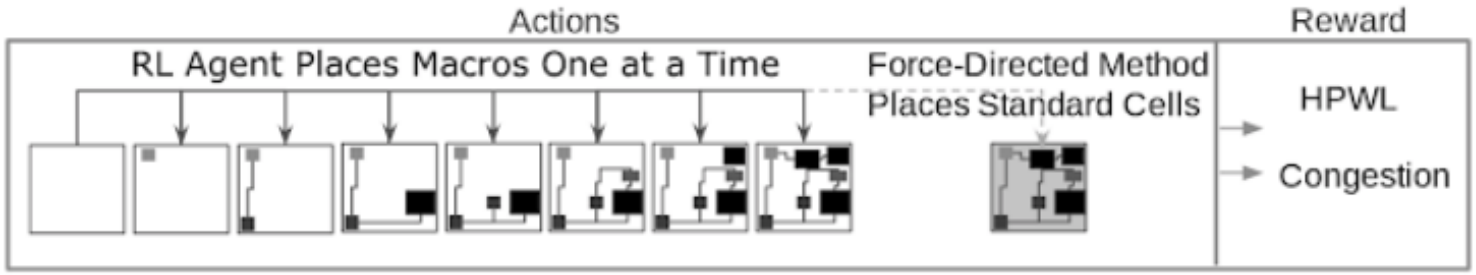
Rufus will take this feedback and use it to update his language model. He will try to say the sentence again, using the new information he received from the human. This time, he might say "I'm a robot."

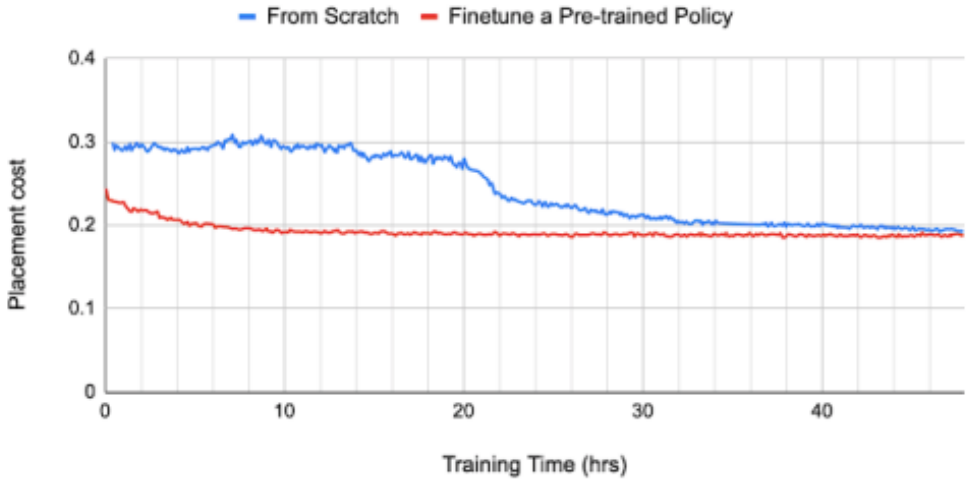
The human will listen again and give Rufus more feedback. This process will continue until Rufus can say sentences that sound natural to a human.

Over time, Rufus will learn how to talk like a human thanks to the feedback he receives from humans. This is how language models can be improved using RL with human feedback.

Source: https://huggingface.co/blog/rlhf

# RL for Chip Design




Source: https://ai.googleblog.com/2020/04/chip-design-with-deep-reinforcement.html

# Main Takeaways

- MDPs assumed that we know an a-priori model of the domain
  - In which case, we can derive the optimal policy. Then the agent can simply follow it.
- What happens when?
  - We do not have the following:
    - Transition function (*if I take an action which state I will end up in?*)
    - Reward function (*when I take a transition is it good or bad?)*
- The agent must interact/explore the environment
  - Learning from experience
    - It observes transitions and rewards as a function of actions
    - Learns the optimal policy from the experience data it collects.